

Week 2 - Day 8a Supplementary: More Recursion!

Exercise 1: Binary Converter [] []

The decimal number system is a common way to write numbers. It is based on the base 10, meaning that each number is represented as the sum of powers of 10. For example, the number 127 is expressed as

$$127 = 1 \cdot 10^2 + 2 \cdot 10^1 + 7 \cdot 10^0$$

Hence, the digits in 127 represent the coefficients of the powers of 10. In the decimal number systems there are 10 digits (0-9).

However, computers use the binary number system, which is based on the base 2. In the binary number system there are only 2 digits (0 and 1). The same number 127 in binary is expressed as

$$127 = 1 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

And, hence the binary string is 1111111.

1.1 Write a recursive function, `to_binary` that takes an integer n as input and returns its binary representation as a string.

```
# TODO: Your code here
def to_binary(n):
    pass

# Use this cell to test your code
assert to_binary(0) == '0'
assert to_binary(1) == '1'
assert to_binary(2) == '10'
assert to_binary(3) == '11'
assert to_binary(4) == '100'
assert to_binary(5) == '101'
assert to_binary(6) == '110'
assert to_binary(7) == '111'
assert to_binary(8) == '1000'
assert to_binary(39) == '100111'
assert to_binary(128) == '10000000'
assert to_binary(297371) == '1001000100110011011'

print("All tests passed. [ ]")
```

1.2 Similarly, we can convert from binary to decimal. Implement the function `to_decimal` recursively that takes a binary string as input and returns its decimal representation.

```

# TODO: Your code here
def to_decimal(b):
    pass

# Test cases for to_decimal function
assert to_decimal('0') == 0
assert to_decimal('1') == 1
assert to_decimal('10') == 2
assert to_decimal('11') == 3
assert to_decimal('100') == 4
assert to_decimal('101') == 5
assert to_decimal('110') == 6
assert to_decimal('111') == 7
assert to_decimal('1000') == 8
assert to_decimal('100111') == 39
assert to_decimal('10000000') == 128
assert to_decimal('1001000100110011011') == 297371

print("All tests passed. ☐")

```

If both functions are correct, we can convert back and forth between decimal and binary without any loss of information.

```

# Additional tests to ensure the functions work together
assert to_decimal(to_binary(12278)) == 12278
assert to_binary(to_decimal('100111')) == '100111'

print("All tests passed. ☐")

```

Exercise 2: Checking Powers of Three [☐☐☐]

Given an integer n , return `True` if it is a power of three, else return `False`. An integer n is a power of three if there exists an integer x such that $n = 3^x$.

```

is_power_of_three(27) == True
is_power_of_three(0) == False
is_power_of_three(-1) == False

# TODO: Your code here
def is_power_of_three(n):
    pass

# When you finish, these should all print True
assert is_power_of_three(27) == True
assert is_power_of_three(8) == False
assert is_power_of_three(0) == False
assert is_power_of_three(-1) == False

print("All tests passed. ☐")

```

```

-----
-----
AssertionError                                Traceback (most recent call
last)
Cell In[4], line 2
      1 # When you finish, these should all print True
----> 2 assert is_power_of_three(27) == True
      3 assert is_power_of_three(8) == False
      4 assert is_power_of_three(0) == False

AssertionError:

```

Exercise 3: Subsets [] []

A subset S of a set A is a collection of elements from A that is itself a set. For example, if $A = \{1, 2, 3\}$, then the subsets of A are:

$$\{\}, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}$$

Implement a function `subsets` that takes a list of integers `nums` and returns a list of all possible subsets of `nums`.

```

# TODO: Your code here
def subsets(nums):
    pass

assert sorted(subsets([6])) == [[], [6]]
assert sorted(subsets([1, 2, 3])) == [[], [1], [1, 2], [1, 2, 3], [1,
3], [2], [2, 3], [3]]
assert sorted(subsets([4, 5])) == [[], [4], [4, 5], [5]]

print("All tests passed. [ ]")

```

Exercise 4: Climbing Stairs [] []

Given an n , such that $n > 0$, return the number of unique ways you can climb n stairs, if you're allowed to climb 1, 2, or 3 stairs at a time. For example,

```

climb_stairs(1) == 1
climb_stairs(2) == 2
climb_stairs(3) == 4

```

Implement `climb_stairs(n)` recursively.

```

# TODO: Your code here
def climb_stairs(n):
    pass

```

```
# Use this code cell to test your solution
assert climb_stairs(1) == 1
assert climb_stairs(2) == 2
assert climb_stairs(3) == 4
assert climb_stairs(4) == 7
assert climb_stairs(10) == 274

print("All tests passed. ☑")
```

Exercise 5: Parentheses [] [] []

Write a recursive function `parentheses` that generates all possible pairs of parentheses with `n` pairs. For example,

```
parentheses(1) == ["()"]
parentheses(2) == ["()", "()()"]
parentheses(3) == ["((()))", "(()())", "(()())()", "()(())", "()(())()"]

# TODO: Your code here
def parentheses(n):
    pass

# Use this cell to test your code
assert set(parentheses(1)) == set(["()"])
assert set(parentheses(2)) == set(["()", "()()"])
assert set(parentheses(3)) == set(["((()))", "(()())", "(()())()", "()(())", "()(())()"])

print("All tests passed. ☑")
```

Exercise 6: Coin Change [] [] []

Given an integer amount and a list of distinct coin values, find the **minimum number of coins needed** to make up that amount. If that amount of money cannot be made up by any combination of the coins, return `-1`. For example,

```
change([1, 2, 5], 11) == 3
change([1, 3, 5], 9) == 3
change([2], 3) == -1
change([1], 0) == 0

# TODO: Your code here
def change(coins, amount):
    pass

# Use this cell to test your code
assert change([1, 2, 5], 11) == 3
assert change([1, 3, 5], 9) == 3
assert change([2], 3) == -1
```

```
assert change([1], 0) == 0  
print("All tests passed. 🎉")
```

Congratulations 🎉